Database Indexing

Introduction

- Many queries reference only a **small portion** of the records in a relation.
- Example queries:
  - "Find all instructors in the Physics department."
  - "Find the total number of credits earned by student with ID 22201."
- Without indices, the database system must perform a **full table scan**, which is highly inefficient.
- **Solution:** Introduce **index structures** to locate records directly.

Basic Concepts

- An index in a database is analogous to the index in a textbook:
  - Quickly locates information based on a keyword (search key).
  - The index is **smaller** than the dataset itself.
  - Entries are **sorted** for efficient lookup.
- In databases:
  - To retrieve a student record by ID, the DBMS uses the index to find the disk block where the record resides.
  - Then it fetches the record from storage.
- **Importance:** Indices drastically reduce query cost compared to scanning all tuples.

Problem with Simple Sorted Index

- Keeping a sorted list of student IDs is **inefficient for large databases**:
  1. Index itself becomes too large.
  2. Search is still relatively expensive.
  3. Updating (insertions/deletions) is costly.
- Therefore, advanced indexing techniques are used.

Types of Indices

There are two broad categories:

1. **Ordered Indices** – Based on a sorted ordering of values.
2. **Hash Indices** – Based on uniform distribution across buckets using a hash function.

Evaluation Criteria for Indexing Techniques

Each indexing technique is evaluated based on:

- **Access Types:** Efficient support for exact match and range queries.
- **Access Time:** Time required to locate data items.
- **Insertion Time:** Time to add a new item (includes locating correct position + updating index).
- **Deletion Time:** Time to remove an item (includes locating + restructuring index).
- **Space Overhead:** Additional memory required by the index structure.

Multiple Indices and Search Keys

- Often, multiple indices are maintained for the same file.
- Example: Searching for a book by **author, subject, or title**.
- **Search Key:** Attribute(s) used to look up records in a file.
  - Different from *primary key*, *candidate key*, or *superkey*.
  - A relation may have several search keys depending on the number of indices created.

Summary

- Indexing is essential for query optimization in databases.
- Two main types: **Ordered** and **Hash**.
- Indices must be designed considering:
  - Access efficiency,
  - Update performance,
  - Storage overhead.
- Multiple indices allow flexible retrieval but increase storage cost.